

```
In [ ... import marimo as mo
```

```
In [ ... import cv2
import matplotlib.pyplot as plt

d50=cv2.imread('a2/50.jpeg',cv2.IMREAD_GRAYSCALE)
d80=cv2.imread('a2/80.jpeg',cv2.IMREAD_GRAYSCALE)
```

Question 1

In Problem 4 of Assignment_01, find out the number of pixels per square inch (DPI) for both the images taken at 50 cm and 80 cm respectively.

```
In [ ... dim_50=d50.shape
dim_80=d80.shape
print(dim_50, dim_80)
# We have 960/1280 images both
```

```
In [ ... # d50_bw = d50.copy()
#print(h,w)
def bw_convert(img,T=90):
    bw = img.copy()
    h, w = img.shape
    for i in range(h):
        for j in range(w):
            if img[i,j] ≥ T:
                bw[i,j] = 255
            else:
                bw[i,j] = 0
    return bw

d50_bw=bw_convert(d50)
# cleaning up
h, w = d50.shape

for i in range (520,h):
    for j in range(w):
        d50_bw[i,j] = 255
for i in range (h):
    for j in range (740,w):
        d50_bw[i,j] = 255
for i in range(h):
    for j in range(0,500):
        d50_bw[i,j] = 255
plt.imshow(d50_bw, cmap='gray')
plt.show()
```

```
In [ ... d80_bw = bw_convert(d80)
# adjust
```

```

for i_1 in range(490, h):
    for j_1 in range(w):
        d80_bw[i_1, j_1] = 255
for i_1 in range(h):
    for j_1 in range(754, w):
        d80_bw[i_1, j_1] = 255
for i_1 in range(h):
    for j_1 in range(0, 600):
        d80_bw[i_1, j_1] = 255
plt.imshow(d80_bw, cmap='gray')
plt.show()

```

```

In [ ... # measuring the area in pixels
def area_count(img):
    circle_area = 0
    for i in range(h):
        for j in range(w):
            if img[i, j] == 0:
                circle_area = circle_area + 1
    return circle_area
ax = area_count(d50_bw)
ay = area_count(d80_bw)
print(f'area of 50cm away circle is {ax} and that of 80cm away circle
is {ay}')

```

Answer

After doing the conversion of area -> radius.

- radius in px/ actual radius in inch
- obtained linear pixel density
- squared to get DPI (pixels per sq inch)

Q2

Put image in aJached document Number_of_Lines_Assignment_02.pdf in matrix form of greyscale intensity. Describe your own algorithm to count number of lines.

Implement your algorithm to count number of lines in the document:

Number_of_Lines_Assignment_02.pdf

```

In [ ... img = cv2.imread('a2/lines.jpg', cv2.IMREAD_GRAYSCALE)
img = bw_convert(img)
h_1, w_1 = img.shape
lin_count = 0
white_check = False
for i_2 in range(h_1):
    black_num = 0
    for j_2 in range(w_1):
        if int(img[i_2, j_2]) == 0:
            black_num = black_num + 1

```

```

        if black_num > 100:
            if not white_check: #this is where it detects first black pixel
                lin_count = lin_count + 1
                white_check = True
            else:
                white_check = False
print(lin_count)

```

Q3

Use different thresholding algorithms on the following image to generate and save the binary image after thresholding.

```

In [ ... polka=cv2.imread('a2/polka.png')
g_polka = cv2.imread('a2/polka.png', cv2.IMREAD_GRAYSCALE)

# Colored thresholding - bg white, color shown
def color_convert(img, T=80):
    color = img.copy()
    h, w, c= img.shape
    for i in range(h):
        for j in range(w):
            for k in range(c):
                if img[i,j,k] ≥ T:
                    color[i,j,k] = 255
                else:
                    color[i,j] = 0
    return color
# black and white thresholding (binary)
bw_polka = bw_convert(g_polka)
color_polka =color_convert(polka)

```

```

In [ ... ### BINARY THRESHOLDING ###

def max_int(img):
    h, w = img.shape
    store=[]
    for i in range(h):
        for j in range(w):
            store.append(img[i,j])
    return max(store)

m=int(max_int(g_polka))
print(m)

```

```

In [ ... ## BINARY THRESHOLD
def conv(x):
    return (2**x-1)
# m predefined

```

```

def binary_thresh(img, p0=55):
    store = img.copy()
    p0 = 50
    h, w = img.shape
    for i in range(h):
        for j in range(w):
            if img[i,j] > p0:
                intensity = conv(img[i,j])
                if intensity ≥ 255:
                    intensity = 255
                store[i,j] = intensity
            else:
                store[i,j] = 0
    return store
binary_output = binary_thresh(g_polka)

plt.imshow(binary_output, cmap='gray')
plt.title(f"Binary Threshold")
plt.show()

```

In [... *### OTSU THRESHOLDING ###*

```

img_2 = cv2.imread('a2/polka.png', cv2.IMREAD_GRAYSCALE)
def plot_histogram(img, title="Pixel Intensity Histogram"):
    ht, wt = img.shape
    freq = []
    for _i in range(256):
        freq.append(0)
    for _i in range(ht):
        for _j in range(wt):
            intensity = img[_i, _j]
            freq[intensity] = freq[intensity] + 1

    intensity = range(256)
    plt.figure(figsize=(10, 5))
    plt.bar(intensity, freq, width=1.0)
    plt.xlabel('Intensity')
    plt.ylabel('Frequency')
    plt.title(title)
    plt.show()
    plt.close()

    return freq
freq = plot_histogram(img_2)

```

In [...

```

In [ ... image = cv2.imread('a2/polka.png', cv2.IMREAD_GRAYSCALE)
def otsu_thresholding_full(image):
    height, width = image.shape

    pixels = image.tolist() # cool fn, instant conversion

```

```

# 2d to 1d - hist
flat_pixels = []
for row in pixels:
    for val in row:
        flat_pixels.append(val)
hist = [0] * 256
for val in flat_pixels:
    hist[val] += 1

# mean total
total_pixels = len(flat_pixels)
sum_total = 0
for i in range(256):
    sum_total += i * hist[i]

#otsu algo
def otsu_thresholding(total_pixels, sum_total):
    current_max_var = 0.0
    threshold = 0
    # bg vals
    weight_bg = 0
    sum_bg = 0

    for t in range(256): # moving from 0 to 156
        weight_bg += hist[t] # bg weight being added
        if weight_bg == 0:
            continue
        # fg wt comes from number of pixels assumed to be in bg, hist
        # cut by t
        # div by all number of pixels in bg
        weight_fg = total_pixels - weight_bg # from the relation
        # If all pixels are in background,
        if weight_fg == 0:
            break
        # bg_sum update
        sum_bg += t * hist[t]

        # mean = sum/wt
        mean_bg = sum_bg / weight_bg
        mean_fg = (sum_total - sum_bg) / weight_fg
        #bw class variance = w0 * w1 * (mu0 - mu1)^2
        var = weight_bg * weight_fg * (mean_bg - mean_fg) ** 2

        # finding the maximum inter class variance
        if var > current_max_var:
            current_max_var = var
            threshold = t
    return int(threshold)

threshold = otsu_thresholding(total_pixels, sum_total)
print(f"Optimal Threshold found: {threshold}")

```

```

binary_image = []
for i in range(height):
    row = []
    for j in range(width):
        row.append(0)
    binary_image.append(row)

for r in range(height):
    for c in range(width):
        if pixels[r][c] > threshold:
            binary_image[r][c] = 255
        else:
            binary_image[r][c] = 0

# Displaying
plt.imshow(binary_image, cmap='gray')
plt.title(f"Otsu threshold, T={threshold}")
return plt.show()

```

```
otsu_thresholding_full(image)
```

```

In [ ... cv2.imshow('title', img_2)
cv2.waitKey(0)
cv2.destroyAllWindows()

```

```

In [ ... cv2.imshow('title', color_polka)
cv2.waitKey(0)
cv2.destroyAllWindows()

```

Q4

Draw a fence of an arbitrary shape by hand (!) on a clean paper with white background and printed lines on it as explained in the class. Take a photo of this image. Write a program:

- to read the photographed image and translate it to a gray scale image of size say, 300 x 300 of this image.
- Use global thresholding algorithm to clean image to remove all unwanted lines on it if any.
- Save original and binary image by name fence_original.jpg and fence_threshold.jpg.
- Write a program to draw a filled circle of radius 5 randomly in this image (fence_threshold.jpg).

Generate 50 such images. Store these images into two folders as explained in the class.

```

In [ ... def bw_convert1(img, T=90):
    bw = img.copy()

```

```

h, w = img.shape
for i in range(h):
    for j in range(w):
        if img[i,j] ≥ T:
            bw[i,j] = 255
        else:
            bw[i,j] = 0
return bw

```

In [...

```

img_3 = cv2.imread('a2/fence.jpg', cv2.IMREAD_GRAYSCALE)
def mean_filtering_full(img_3):
    pixels = img_3.tolist()
    # Input: 1280 | Output: 300
    # 300 = (1280 - f)/s + 1
    stride = 4
    f_size = 84 # Filter size
    output_dim = 300

    # 300x300 output
    output_img = []
    for i in range(output_dim):
        row = []
        for j in range(output_dim):
            row.append(0)
        output_img.append(row)

    # Mean filter
    def mean_filter(pixels):
        for r in range(output_dim):
            for c in range(output_dim):

                # Determine where the window starts on the big image
                start_y = r * stride
                start_x = c * stride
                current_sum = 0
                # Loop through the 84x84 filter window
                for ky in range(f_size):
                    for kx in range(f_size):
                        # Get the pixel value
                        val = pixels[start_y + ky][start_x + kx]
                        current_sum += val
                # Avg find out = (Total Sum / Total Pixels in Filter)
                average = current_sum // (f_size * f_size)

                output_img[r][c] = average
    return output_img

out_img = mean_filter(pixels)
return(out_img)

x=mean_filtering_full(img_3)
plt.imshow(x, cmap='gray')
plt.gca()

```

```
In [ ... import random

img = cv2.imread("a2/img_gen.png", cv2.IMREAD_GRAYSCALE)
h, w = img.shape
radius = 5
def random_circle_generator(img):
    for i in range(50):
        img_circle = img.copy()
        x = random.randint(radius, w - radius - 1)
        y = random.randint(radius, h - radius - 1)
        cv2.circle(img_circle, (x, y), radius, 0, -1)
        cv2.imwrite(f"with_circle/img_{i+1}.jpg", img_circle)
```

Q5 Histogram Equalization

Write a histogram equalization code to improve the contrast of following grayscale image.

Tasks

- A) Show and save input and output (after histogram equalization) grayscale image.
- B) Show and save intensity histogram of input and output image.
- C) Show and save differential probability histogram of input and output image.
- D) Superimpose the cumulative probability histogram on C); the differential probability histogram of input and output image.
- E) Obtain and store the mean intensity of input and output image.

```
In [ ... img_5 = cv2.imread('a2/mri.png', cv2.IMREAD_GRAYSCALE)
mri_hist = plot_histogram(img_5, 'MRI-input hist')
```

```
In [ ... ## Histogram Equalization

# creating hist
def histogram_equalization(img):
    h, w = img.shape
    tot_pixels = h * w
    hist = [0] * 256
    for i in range(h):
        for j in range(w):
            intensity = img[i, j]
            hist[intensity] += 1

    ## prob dist
    prob_hist = [hist[i] / tot_pixels for i in range(256)]
    ## CDF
    cdf = [0] * 256
    cdf[0] = prob_hist[0]
```

```

    for i in range(1, 256):
        cdf[i] = cdf[i-1] + prob_hst[i]
    #intensity mapping
    mapping = [0] * 256
    for i in range(256):
        mapping[i] = int(cdf[i] * 255)

    #old int to new int using CDF
    equalized_img = []
    for i in range(h):
        row = []
        for j in range(w):
            row.append(mapping[img[i,j]])
        equalized_img.append(row)
    return equalized_img

eq_img = histogram_equalization(img_5)
plt.imshow(eq_img, cmap = 'gray')

```

```

In [ ... import numpy as np
eq_img_out = np.array(eq_img, dtype='uint8')
eq_img_hist = plot_histogram(eq_img_out, 'equalized histogram')

```

```

In [ ... # Differential prob histogram
def diff_prob_hist(img):
    h, w = img.shape
    tot_pixels=h*w
    # histogram
    hist = [0] * 256
    for i in range(h):
        for j in range(w):
            intensity = img[i,j]
            hist[intensity]+=1

    #prob dist
    prob_hist = [hist[i]/tot_pixels for i in range(256)]

    return prob_hist

# OF OUTPUT
prob_result = diff_prob_hist(eq_img_out) # Changed variable name
plt.bar(range(256), prob_result)
plt.xlabel('Intensity')
plt.ylabel('Probability')
plt.title("Equalized Differential Probability Histogram")

```

```

In [ ... prob_result_input = diff_prob_hist(img_5) # Changed variable name
plt.bar(range(256), prob_result_input)
plt.xlabel('Intensity')
plt.ylabel('Probability')
plt.title("Equalized Differential Probability Histogram")

```